TriageBot: Evidence-Grounded Multi-Agent Triage of SAST Alerts

1. Team and Project Information

Team Name: G1T4

Darren --> Static-analysis integration, alert normalisation/routing, and benchmark preprocessing
Wei Lin --> Security-knowledge retrieval/RAG and source-code context retrieval
Hong Yi --> Multi-agent council and LLM orchestration
Lisa --> Deterministic validation and human-review workflow
Bradley --> Evaluation harness, baselines, metrics, and experimental analysis

2. Problem Statement

Static Application Security Testing (SAST) tools identify potentially vulnerable code, but some findings are false positives that still require manual investigation. Developers and application-security teams may need to inspect surrounding code, data flow, defensive checks, and configuration before deciding whether an alert is genuine. Reviewing every alert consumes engineering effort, while dismissing alerts too aggressively risks hiding real vulnerabilities.

In our initial run (Semgrep 1.178.0, p/java ruleset), Semgrep flagged 1,515 of the Benchmark's 2,740 test cases, of which 520 (34%) were false positives according to the official answer key.

TriageBot improves this existing software-engineering practice by operating as a post-processing layer after a conventional SAST scanner. For each scanner finding, it gathers relevant evidence and returns **likely real vulnerability, likely false positive, or human review required**, together with supporting evidence. TriageBot does not replace the scanner or attempt to discover vulnerabilities that the scanner missed.

3. Justification for Using AI

The conventional approach combines static-analysis rules with manual security review. Deterministic rules work well when facts can be checked exactly, but triage may require contextual reasoning across several pieces of code and security guidance. An LLM is suitable for these reasoning-heavy cases because it can interpret source code and natural-language guidance together, combine evidence, and explain its conclusion.

TriageBot uses the **same underlying LLM in three specialised roles**. Both Analyst and Skeptic receive a common evidence pack containing the scanner alert, relevant code context, vulnerability metadata, and retrieved OWASP guidance. The **Analyst** uses code-exploration tools to investigate evidence supporting a genuine vulnerability, while the **Skeptic** searches for defensive checks, overwritten values, unreachable conditions, or other evidence suggesting a false positive. The **Chair** receives both evidence-backed reports and applies an explicit decision rubric. A deterministic validator then checks factual claims that can be verified directly against the source code. We deliberately use one model for all roles so that the comparison with a single agent isolates the effect of role decomposition; mixing model families is a planned extension.

4. Proposed Solution and Technical Approach

[TriageBot Architecture Diagram.drawio.svg]
Source Code → Semgrep → Normalised Alert → Common Evidence → Analyst / Skeptic Specialist Exploration → Chair → Deterministic Validator → Triage Outcome

The common-evidence layer retrieves relevant source-code context and applicable OWASP security guidance. Role-specific lightweight code-exploration tools allow the Analyst and Skeptic to investigate the same finding from different objectives without attempting to build a full static-analysis engine. Unsupported or uncertain conclusions are escalated to human review.

Planned technology: Python for orchestration and evaluation; **Semgrep Community Edition** for static analysis; **OpenRouter** for hosted LLM access using Gemini 3.8 Flash; **SentenceTransformers + FAISS** for planned semantic retrieval of OWASP guidance; OWASP security resources as the knowledge base; and **OWASP Benchmark Java v1.2** as the evaluation dataset. Initial feasibility testing has confirmed Semgrep scanning, Benchmark answer-key mapping, OpenRouter access, small-sample council execution, lightweight evidence extraction, and basic citation validation. Embedding-based retrieval and stronger deterministic validation remain planned implementation work.

5. Evaluation Plan

The **OWASP Benchmark Java v1.2** provides **2,740 labelled test cases** and an official answer key, allowing TriageBot's outputs to be scored against ground truth. The answer key will remain hidden from all LLM agents and will be used only by the evaluation harness.

We will compare **A)** scanner only, **B)** a single LLM agent with the combined evidence and tools available to both specialists and a comparable token budget, **C)** the Analyst–Skeptic–Chair council, and **D)** the council plus deterministic validation. This controls for evidence access and allows us to test whether specialist decomposition itself provides value.

Primary metrics are **genuine scanner findings retained, false-positive alerts removed, genuine vulnerabilities wrongly dismissed, human-review rate, and per-category performance**. Latency and token usage will also be recorded. Initial success means retaining **≥95% of genuine scanner findings** while removing **≥25% of false-positive alerts**, without escalating the majority of findings to human review.

Because the Benchmark is public, comments and answer-revealing hints will be removed from the LLM-facing copy while preserving the original scanner input. Selected mutation tests will also modify small code conditions so that the correct verdict changes, helping test whether decisions respond to code evidence rather than memorised benchmark patterns.

6. Checkpoint Targets

By **18 October**, the team will:

- run Semgrep on the OWASP Benchmark, map findings to the official answer key, and produce the conventional baseline scorecard;
- connect the main pipeline from scanner finding through common evidence, specialist exploration, council decision, validation, and final output, with incomplete components clearly documented as simplified or stubbed;
- demonstrate at least one reasoning-heavy vulnerability category end-to-end with working security-guidance retrieval and an initial Analyst–Skeptic–Chair workflow;
- evaluate approximately **100 scanner findings**, including a preliminary **single-agent versus multi-agent comparison on the same cases**, and report initial metrics and failure cases; and
- maintain a shared repository with setup instructions, contributions from all members, and the required AI-use disclosure log.